实验 6　Linux 进程间通信—消息队列

1. 实验目的

了解消息队列的通信机制及原理，掌握消息通信相关系统调用的使用方法。

2. 相关知识与系统调用

消息的创建、发送和接收。多个进程通过访问一个公共的消息队列来交换信息。

消息队列：即消息的一个链表。

任何进程都可以向消息队列中发送消息(消息类型及正文)，其他进程都可以从消息队列中根据类型获取相应的消息。

(1) 头文件：“sys/msg.h”

(2) 打开或创建消息队列：int msgget(key_t key, int msgflg);

key：消息队列的键

IPC_PRIVATE：创建一个私有的消息队列

其他：可被多个进程使用的消息队列

msgflg：设置操作类型及访问权限 IPC_CREAT / IPC_EXCL

(3) 获得或设置消息队列属性：int msgctl(int msgid, int cmd, struct msqid_ds *data);

(4) 发送消息：int msgsnd(int msgid, const void *msgp, size_t msgsize, int flags);

参数：

msgid：消息队列标识符 id

msgp：指针，用户自定义缓冲区，可定义成结构体类型，包含两项　long mtype，代表消息类型，char mtext[MTEXTSIZE]; 消息正文

msgsize：要发送消息正文的长度

mflags：标志，若设置 IPC_NOWAIT 则不等待消息发出就返回

返回值：成功：返回 0

错误：返回-1 (置 errno)

(5) 接收消息：int msgrcv(int msgid, void *msgp, size_t mtexsize, long msgtype,　int flags);

参数：与 msgsnd 类似

msgtype：　>0：只接收指定类型消息的第一个

=0：不管什么消息类型都读取队列中第一个数据

<0：接收等于或小于其绝对值的最低类型的第一个，如有 5、6、17 三类，若为-6，则获取类型 5 的。

返回值：成功：返回消息正文字节数

错误：返回-1 (置 errno)

3. 实验程序

【程序 3_14】一个进程内部的消息通信，用来检验 msgtype 的作用。

```
1.  #include <sys/types.h>
2.  #include <sys/msg.h>
3.  #include <unistd.h>
4.  #include <stdio.h>
5.  #include <stdlib.h>
6.  #include <string.h>
```

```
7.
8.  void main(){
9.
10.     int msgid; int status;
11.     char str1[ ]={"test message:hello!"};
12.     char str2[ ]={"test message:goodbye!"};
13.     char str3[ ]={"The third message!"};
14.
15.     struct msgbuf{
16.         long msgtype;
17.         char msgtext[1024];
18.     }sndmsg,rcvmsg;
19.
20.     if((msgid = msgget(IPC_PRIVATE,0666)) == -1){
21.         printf("msgget error!\n");
22.         exit(254);
23.     }
24.
25.     sndmsg.msgtype = 111;
26.     sprintf(sndmsg.msgtext,str1);
27.
28.     if(msgsnd(msgid,&sndmsg,sizeof(str1)+1,0) == -1){
29.         printf("msgsnd error!\n");
30.         exit(254);
31.     }
32.
33.     sndmsg.msgtype = 222;
34.     sprintf(sndmsg.msgtext,str2);
35.
36.     if(msgsnd(msgid,&sndmsg,sizeof(str2)+1,0) == -1){
37.         printf("msgsnd error!\n");
38.         exit(254);
39.     }
40.
41.     sndmsg.msgtype = 333;
42.     sprintf(sndmsg.msgtext,str3);
43.
44.     if(msgsnd(msgid,&sndmsg,sizeof(str3)+1,0) == -1){
45.         printf("msgsnd error!\n");
46.         exit(254);
47.     }
48.
49.     if(status = msgrcv(msgid,&rcvmsg,80,222,IPC_NOWAIT) == -1){
50.         printf("msgrcv error!\n");
```

```
        exit(254);
    }

    printf("The received message:%s.\n",rcvmsg.msgtext);
    msgctl(msgid,IPC_RMID,0);
    exit(0);
}
```

【程序运行结果截图】

```
The received message:test message:goodbye!.
```

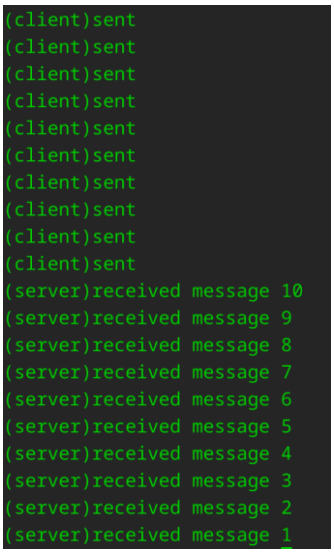
程序思考：将上述程序中的 msgrcv(msgid,&rcvmsg,80,222,IPC_NOWAIT)这句里的 222 分别换成 333 和 0，来分别再做做，观察运行结果有什么变化？

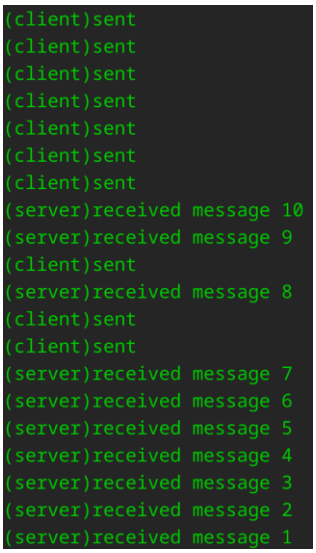
【程序 3_15】不同进程间的消息通信。通信的进程间通过 MSGKEY 值来确定消息队列。

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/msg.h>
#include <sys/ipc.h>
#include <stdlib.h>
#include <unistd.h>
#include <wait.h>

#define MSGKEY 75

struct msgform{
    long mtype;
    char msgtext[1030];
}msg;

int msgqid, pid, pid1;

void SERVER(){
    do{
        msgrcv(msgqid,&msg,1030,0,0);
        printf("(server)received message %d\n",msg.mtype);
    }while(msg.mtype != 1);
    msgctl(msgqid,IPC_RMID,0);
    exit(0);
}

void CLIENT(){
    int i;
    msgqid = msgget(MSGKEY,0777);
```

```
    for(i = 10; i >= 1; i--){
        msg.mtype = i;
        printf("(client)sent\n");
        msgsnd(msgqid,&msg,1030,0);
    }
    exit(0);
}

void main(){
    msgqid = msgget(MSGKEY,0777|IPC_CREAT);
    while((pid = fork()) == -1);
    if(pid == 0)
        SERVER();
    while((pid1 = fork()) == -1);
    if(pid1 == 0)
        CLIENT();
    wait(0);
    wait(0);
}
```

【程序运行结果截图】





【程序 3_16】在【程序 3_15】的基础上，发送和接收有内容的消息。

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/msg.h>
#include <sys/ipc.h>
#include <stdlib.h>
#include <unistd.h>
#include <wait.h>
#include <string.h>
```

```
9.
10. #define MSGKEY 75
11.
12. struct msgform{
13.     long mtype;
14.     char msgtext[1030];
15. }msg;
16.
17. int msgqid, pid, pid1;
18.
19. void CLIENT(){
20.     int i;
21.     char string_i[10];   //存放信息
22.     msgqid = msgget(MSGKEY,0777);   //打开一个消息队列，0777 是文件的存取权限
23.     for(i = 10; i >= 1; i--){
24.         msg.mtype = i;
25.         printf("(client)sent #%d\n", i);
26.         sprintf(msg.msgtext,"the content of message is:  ");
27.         sprintf(string_i,"message  %d",i);
28.         strcat(msg.msgtext,string_i);
29.         strcat(msg.msgtext,"\n");
30.         msgsnd(msgqid, &msg, 1030, 0);
31.     }
32.     exit(0);
33. }
34.
35. void SERVER(){
36.     do{
37.         msgrcv(msgqid,&msg,1030,0,0);
38.         printf("(server)received message %d,  ",msg.mtype);
39.         printf("%s",msg.msgtext);
40.     }while(msg.mtype != 1);
41.     msgctl(msgqid,IPC_RMID,0);
42.     exit(0);
43. }
44.
45.
46.
47. void main(){
48.     msgqid = msgget(MSGKEY,0777|IPC_CREAT);
49.     while((pid = fork()) == -1);
50.     if(pid == 0)
51.         SERVER();
52.     while((pid1 = fork()) == -1);
```

```
53.     if(pid1 == 0)
54.         CLIENT();
55.     wait(0);
56.     wait(0);
57. }
```

【程序运行结果截图】

```
(client)sent
(client)sent
(client)sent
(client)sent
(client)sent
(client)sent
(client)sent
(client)sent
(client)sent
(client)sent
(server)received message 10, the content of message is:  message  10
(server)received message 9, the content of message is:  message  9
(server)received message 8, the content of message is:  message  8
(server)received message 7, the content of message is:  message  7
(server)received message 6, the content of message is:  message  6
(server)received message 5, the content of message is:  message  5
(server)received message 4, the content of message is:  message  4
(server)received message 3, the content of message is:  message  3
(server)received message 2, the content of message is:  message  2
(server)received message 1, the content of message is:  message  1
```

4. 实验思考与扩充

将【程序 3_16】和软中断程序结合，实现客户端每发送一条消息，服务器端接受一条消息。